Rapport— Projet Clip

Belguith Ahmed Rami — 22516111
Benboubaker Majdi — 22517042

Master 2 - Université de Montpellier

1 Introduction

Ce projet implémente un modèle CLIP (Contrastive Language-Image Pre-training) permettant l'alignement de représentations visuelles et textuelles dans un espace latent commun. Le système réalise une recherche bidirectionnelle : texte→images et image→textes.

Notre pipeline se décompose en trois phases : (1) classifieurs indépendants pour images et textes, (2) construction du modèle CLIP par alignement contrastif, et (3) système d'inférence multimodal.

Nous avons tenté d'industrialiser le code (notamment à travers le fichier utils.py). Cependant, en raison de divergences de versions des bibliothèques et des contraintes liées au travail en groupe, l'avancement du projet a été limité. En conséquence, l'implémentation finale repose principalement sur des notebooks.

2 Données et prétraitement

Dataset Flickr : 800 paires image-texte, 4 classes équilibrées (`ball`, `bike`, `dog`, `water`), 200 images par classe. Images : 224×224 pixels, format JPG. Captions enrichies dans `captions.csv`. Split : 80% train / 20% validation.

3 Classifieur d'images

Architecture CNN : 3 blocs Conv2D (32, 64, 128 filtres) + MaxPooling2D, Flatten, Dense(256) + Dropout(0.2), sortie Dense(4) + Softmax.

Hyperparamètres : batch_size=32, Adam (lr=10^{-3}), categorical crossentropy, early stopping (patience=2).

Résultats :

— Best val accuracy : **38.12%**
— Best val loss : **1.3574**
— Epochs : 3 (early stopping)

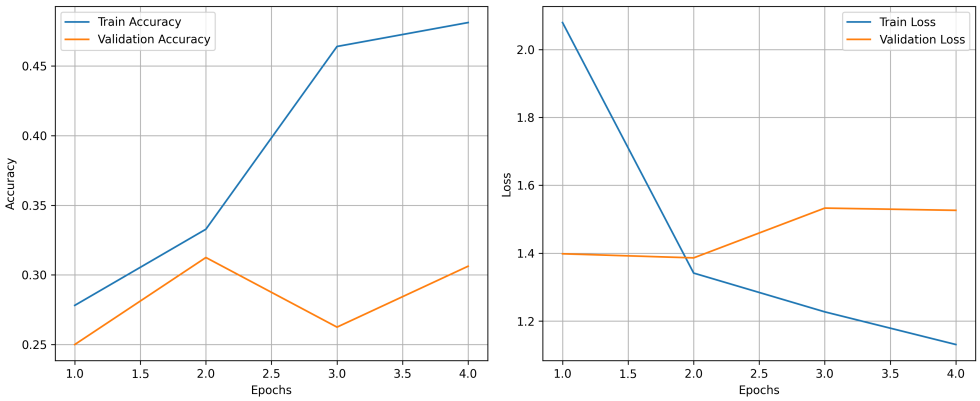


FIGURE 1 – Évolution accuracy/loss - classifieur images

Les performances modestes sont attendues (architecture simple, pas d'optimisation). L'objectif est de créer un encodeur réutilisable, non un classifieur optimal.

4 Classifieur de textes

Architecture SmallBERT :
— Positional Embedding (vocab=10000, seq_len=32)
— 2 TransformerBlocks (4 têtes, embed_dim=128, ff_dim=256)
— LayerNorm + Dropout(0.1)

Pooling : SmallBERT n'a pas de token `[CLS]`. Nous utilisons `MaskedMean` qui calcule la moyenne des embeddings en excluant les PAD :

$$pooled = \frac{\sum_{i=1}^{L} mask_i \cdot emb_i}{\sum_{i=1}^{L} mask_i}$$

Tokenisation : TextVectorization (vocab=10000, seq_len=32, standardize=lowercase+remove punctuation). Vocabulaire sauvegardé dans `vocab.txt`.

Résultats :
— Val accuracy : **80.00%**
— Val loss : **0.6124**
— Epochs : 8

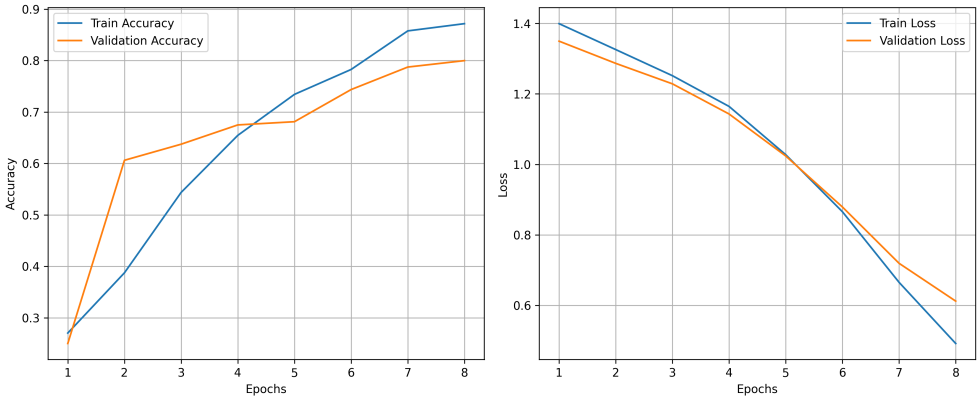


FIGURE 2 – Accuracy/loss - classifieur textes

Les Transformers capturent mieux les dépendances contextuelles (80% vs 38% pour les images).

5 Construction des encodeurs CLIP

À partir des classifieurs entraînés, nous dérivons deux encodeurs en supprimant les têtes de classification :

Encodeur image :

- Extraction jusqu'à la couche `flatten`
- Projection : Dense(128, activation=None)
- Normalisation L2 (norme=1)

Encodeur texte :

- Extraction jusqu'à `masked_mean`
- Projection : Dense(128, activation=None)
- Normalisation L2 (norme=1)

La normalisation L2 est cruciale : elle permet d'utiliser le produit scalaire comme mesure de similarité cosinus.

6 Modèle CLIP

6.1 Perte contrastive

Pour un batch de B paires (image$_i$, texte$_i$), nous calculons la matrice de similarité $S \in R^{B \times B}$:

$$S_{ij} = \frac{emb_{img}^{(i)} \cdot emb_{txt}^{(j)}}{\tau}$$

La perte contrastive rapproche les paires correctes (diagonale) et éloigne les paires

incorrectes. Implémentée via `ClipLossLayer` avec `add_loss()`, évitant les complications de `compile(loss=...)`.

6.2 Architecture et entraînement

Split des données : Dataset divisé en train (80%) / validation (10%) / test (10%). Le tokenizer est adapté uniquement sur l'ensemble d'entraînement pour éviter le data leakage.

Dataset CLIP : Pipeline `tf.data` optimisé, batch_size=32 (carré requis),

drop_remainder=True, cache activé. Format : `{"image_input": (32,224,224,3), "text_input": (32,32)}`.

Hyperparamètres : Adam (lr=10^{-4}), epochs=10, batch_size=32, temperature=0.07, early stopping (patience=4, monitor=val_loss).

Résultats :

- Epoch 1 : train_loss=4.20, val_loss=3.49
- Epoch 2 : train_loss=3.42, val_loss=3.47 (meilleur)
- Epoch 6 : early stopping (surapprentissage détecté)
- Test loss : **3.35** (généralisation correcte)

Le modèle converge rapidement (réduction 79% de la loss en 2 epochs). L'early stopping à l'epoch 6 évite le surapprentissage visible dans la divergence train/val.

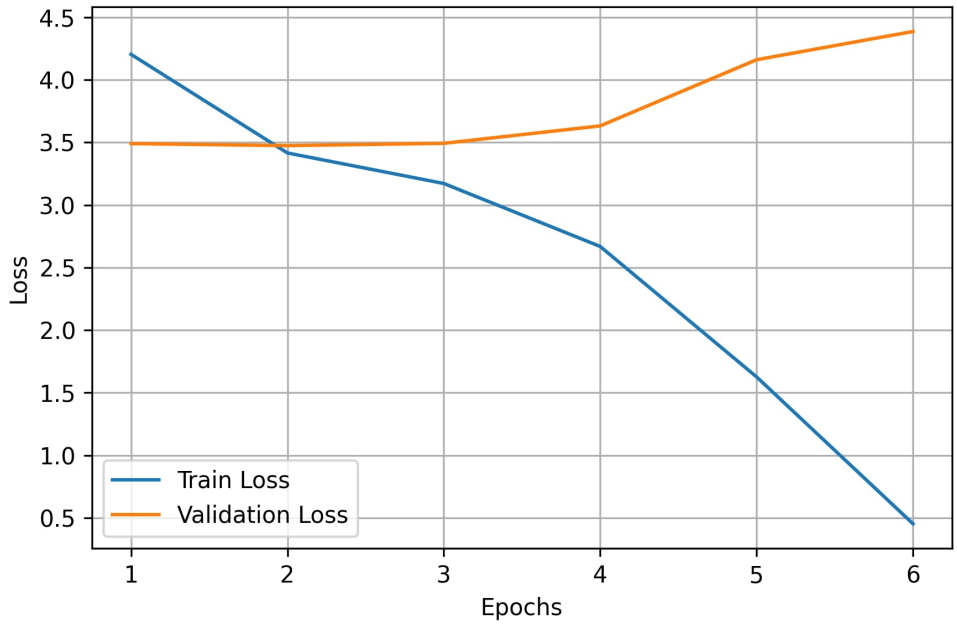


FIGURE 3 – Train/Vlaidation loss - Modéle CLIP

7 Inférence et recherche

7.1 Index d'embeddings

Pré-calcul des embeddings pour recherche efficace :
- **Index images :** (800, 128), `image_index.npz`
- **Index textes :** (800, 128), `text_index.npz`

7.2 Recherche texte → images

Le processus de recherche d'images à partir d'une requête textuelle exploite la propriété d'alignement de l'espace latent CLIP. Voici le pipeline détaillé :

Étape 1 - Prétraitement textuel :
- Normalisation : lowercase + suppression ponctuation
- Tokenisation via le vocabulaire pré-appris (10k tokens)
- Padding/truncation à 32 tokens
- Output : tensor (1, 32) d'indices de tokens

Étape 2 - Encodage : La requête tokenisée passe dans `text_encoder` :
- SmallBERT (2 TransformerBlocks) → (1, 32, 128)
- MaskedMean pooling → (1, 128)
- Dense projection + L2 norm → **(1, 128)** normalisé

Étape 3 - Calcul des similarités : Produit matriciel avec l'index images pré-calculé :

$$scores = q \cdot E_{img}^T$$

où $q \in R^{1\times128}$ (requête) et $E_{img} \in R^{800\times128}$ (index).

Résultat : vecteur (800,) de scores $\in [-1, 1]$. Grâce à la normalisation L2, le produit scalaire équivaut à la similarité cosinus.

Étape 4 - Sélection Top-k :
- `np.argsort(-scores)` : tri décroissant
- Extraction des k=5 premiers indices
- Récupération des chemins d'images correspondants

Exemple : "A dog playing in water"

Scores : 0.4091, 0.4053, 0.4021, 0.3817, 0.3815

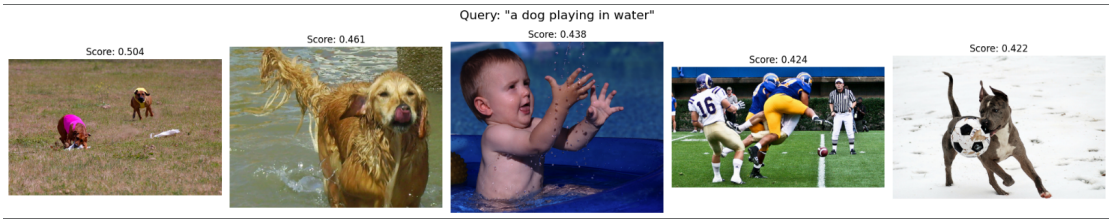


FIGURE 4 – Exemple de recherche de captions à partir d'une image requête (texte → images).

7.3 Recherche image → textes (image-to-caption retrieval)

Cette étape vise à associer à une image requête I_q les descriptions textuelles (captions) les plus pertinentes présentes dans le corpus. Le processus suit un schéma de *retrieval* basé sur des représentations vectorielles apprises (embeddings), plutôt qu'une comparaison pixel à pixel.

Méthode. Soit $f_{\text{img}}(\cdot)$ l'encodeur d'images et $f_{\text{txt}}(\cdot)$ l'encodeur de texte. On procède comme suit :

1. **Prétraitement** : l'image est redimensionnée en 224×224 et normalisée dans $[0, 1]$.
2. **Encodage** : l'image est projetée en un vecteur $q = f_{\text{img}}(I_q) \in R^D$ (dans nos expériences, $D = 128$).
3. **Index texte** : chaque caption c_i est encodée en $e_i = f_{\text{txt}}(c_i) \in R^D$, formant une matrice $E_{\text{txt}} \in R^{N \times D}$.
4. **Similarité** : un score est calculé par produit scalaire (équivalent à une similarité cosinus si les vecteurs sont normalisés) :

$$\text{scores} = q\, E_{\text{txt}}^{\top} \in R^N.$$

5. **Sélection Top-k** : on retourne les k captions ayant les scores les plus élevés, ainsi que les chemins d'images associés dans le CSV pour visualiser les correspondances.



FIGURE 5 – Exemple de recherche de captions à partir d'une image requête (image → textes).

Résultats observés et limites. Pour l'image requête `bike_001.jpg`, les deux premières captions retournées décrivent correctement une scène de vélo (scores $\approx$ 0.44 et 0.36), ce qui indique que le modèle capte une partie de la sémantique globale (présence d'un cycliste/vélo). En revanche, à partir du rang 3, le système retourne des captions sans rapport direct (chien, balle, plage), avec des scores relativement proches de ceux du rang 2 (par exemple 0.345–0.342). Cette proximité des scores entre résultats pertinents et non pertinents traduit une séparation insuffisante dans l'espace des embeddings, et donc des performances de recherche limitées.

8 Conclusion

Pipeline CLIP complète implémentée avec succès : classifieurs indépendants, encodeurs dérivés, perte contrastive, système de recherche multimodal. Compréhension des défis du deep learning multimodal : gestion données hétérogènes, équilibre modalités, importance normalisation L2.

A Configuration technique

Environnement : Google Colab Pro (Tesla T4), Python 3.10, TensorFlow 2.15.0, CUDA 12.2. Temps total : 35 min.

Environnement : PC MSI GeForce GTX 1650 Ti, processeur compatible, Python 3.10, TensorFlow 2.15.0, CUDA 12.9

Récapitulatif hyperparamètres :

- Image CNN : batch=32, lr=10^{-3}, epochs=8 (stopped@3)
- Text SmallBERT : batch=64, lr=10^{-4}, vocab=10k, seq_len=32, epochs=8
- CLIP : batch=32, lr=10^{-4}, temp=0.07, epochs=5
- Embed dim : 128 (images et textes)